

GitHub: <https://github.com/mr-uyghur/tutorconnect-cmpe172-milestone1>

Video: https://drive.google.com/file/d/1u3FaMFmUwonOc5w_u8asEpUoQraFAZew/view?usp=sharing

Milestone 1 - Proposal

Ali Halmamat | TutorConnect is a tutoring appointment system. Students find one-hour sessions with tutors for subjects such as Java and SQL. Tutors offer time slots and see their bookings.

Roles, entities, and technology

Customer: student. **Provider/admin:** tutor. The five main entities are users, providers, services, availability_slots, and appointments. The project uses Java, Spring Boot, and SQL through JDBC; it uses no ORM. It realizes classic enterprise standards on a modern stack: Spring Boot for J2EE-style applications and REST for the role of CORBA/distributed objects.

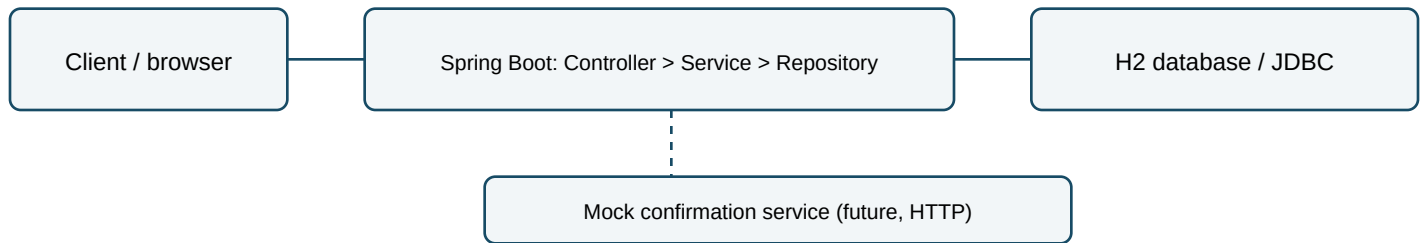
Twelve planned features

1. Student login.
2. Browse, filter by tutor/subject/date, and paginate available slots with SQL LIMIT/OFFSET.
3. Book a slot for its tutoring service.
4. View my upcoming and past appointments.
5. Cancel my own appointment.
6. Tutor login.
7. Tutor creates and removes service-linked availability slots.
8. Tutor views appointments booked with them.
9. Send booking and cancellation confirmations through a mock external service.
10. Add logging, a health endpoint, and one metric.
11. Add a conversational booking agent and an autonomous agent.
12. Add grounded question answering over a small knowledge base.

Milestone plan: Features 1-8 are for Milestone 2, 9-10 for Milestone 3, and 11-12 for Milestone 4. This first milestone implements the layered skeleton and two database reads only.

Architecture and relational schema

Block diagram and request flow



For **GET /slots**, Spring MVC routes the request to `SchedulingController`. It calls `SchedulingService`, which asks `SlotRepository` to run the SQL query. The query selects future slots with no `BOOKED` appointment. The repository fills `SlotDto` objects, which return through the service and controller as JSON. **GET /** follows the same path, using `CatalogRepository` to read tutors and services into `HomeDto`.

Page Controller vs Front Controller

A Page Controller organizes code around individual server-rendered pages, such as Thymeleaf pages. A Front Controller receives requests in one place and dispatches them. TutorConnect uses Spring MVC `DispatcherServlet` as the Front Controller for JSON requests; the four wireframes show planned client screens.

Relational schema (PK = primary key; FK = foreign key)

users: `user_id` PK; `full_name`; `email` UNIQUE; `password_hash` (nullable); `role` CUSTOMER/PROVIDER.

providers: `provider_id` PK; `user_id` FK → `users.user_id` and UNIQUE; `bio`.

services: `service_id` PK; `name` UNIQUE; `description`.

availability_slots: `slot_id` PK; `provider_id` FK → `providers.provider_id`; `service_id` FK → `services.service_id`; `starts_at`; `ends_at`. UNIQUE(`provider_id`, `starts_at`).

appointments: `appointment_id` PK; `customer_id` FK → `users.user_id`; `slot_id` FK → `availability_slots.slot_id`; `status` BOOKED/CANCELLED; `created_at`; `generated_active_slot_id` UNIQUE.

Each slot begins on a whole hour and ends exactly one hour later. All fields are required except `password_hash` and the `generated_active_slot_id`. The full SQL is in *schema.sql*; the ER diagram is a separate PDF.

Database rules and Milestone 1 code

Relationships and keys

A user can have zero or one provider profile. Each provider and each service can have many slots; each slot has one provider and one service. A student can have many appointments, and each appointment belongs to one student and one slot. A slot can have several past appointment records, but only one active booking. Every table has its own primary key, so there are no classical weak entities.

How double-booking is prevented

For a BOOKED appointment, the generated **active_slot_id** equals its slot_id. UNIQUE(active_slot_id) rejects a second active booking for that slot, even if two requests arrive together. For a CANCELLED appointment, active_slot_id is NULL; H2 allows multiple NULLs in a unique column. The old row remains as history and the slot becomes bookable again. A future booking endpoint must turn a duplicate-key error into a clear conflict response.

Tutor schedule rule

Slots start on whole hours and last exactly one hour. UNIQUE(provider_id, starts_at) therefore prevents a tutor from offering overlapping slots, even for different services. Different tutors may offer sessions at the same time.

What runs in this milestone

Spring loads *schema.sql* and *seed.sql* at startup. The sample data has one student, two tutors, two services, and four future slots; slot 2 is already booked. **GET /** returns tutor and service DTOs. **GET /slots** returns slots 1, 3, and 4 as DTOs. The four required wireframes (Home, available slots, booking form, confirmation) are in *Wireframes.pdf*. Booking, login, notifications, and AI are planned features, not implemented Milestone 1 features.

Requirement source: SJSU CMPE 172, *Milestone 1 - Requirements, Design & Skeleton*, supplied two-page assignment PDF, pp. 1-2.